

Université Paris 1 Panthéon-Sorbonne

Master MoSEF — Deep Learning / NLP

Analyse de Sentiment sur Textes Financiers

Benchmarking Classical ML, Deep Learning et Transformers

Houssem MAJED

Victoria VIDAL

Avril 2026

Résumé

Ce rapport présente un projet de comparaison de six modèles de classification de sentiment appliqués à des textes financiers. Nous avons travaillé sur le FinancialPhraseBank, un jeu de données de 4 846 phrases annotées par des experts du domaine. Notre objectif était d'évaluer trois générations d'approches NLP (classiques, deep learning, transformers) sur un même protocole expérimental, puis de construire un dashboard interactif de démonstration. FinBERT, un transformer pré-entraîné sur un corpus financier, obtient les meilleurs résultats avec un macro-F1 de 87,7%, confirmant que la spécialisation domaine est le facteur déterminant sur un petit dataset.

Table des matières

1	Introduction	3
2	Dataset	3
2.1	FinancialPhraseBank	3
2.2	Distribution des classes	4
2.3	Split des données	5
3	Modèles développés	5
3.1	Approche classique : TF-IDF + classifieurs linéaires	5
3.2	Deep Learning : BiLSTM	5
3.3	Transformers : BERT, DistilBERT, FinBERT	6
4	Résultats et benchmark	6
4.1	Tableau comparatif	6
4.2	Leaderboard interactif	7
4.3	Analyse des résultats	7
4.4	Enseignements	8
5	Dashboard et fonctionnalités	8
5.1	Architecture du dashboard	8
5.2	Analyse individuelle : saisie textuelle et saisie vocale	8
5.2.1	Saisie textuelle	8
5.2.2	Saisie vocale	9
5.3	Comparaison multi-modèles et rôle de FinBERT	9
5.4	Explicabilité avec LIME	10
5.5	Batch Analysis	11
5.6	Live News via Yahoo Finance	11
6	Difficultés rencontrées	11
7	Conclusion	12

1. Introduction

Les marchés financiers génèrent chaque jour un volume considérable de textes sous forme de communiqués de presse, de rapports d’analystes, de dépêches Reuters ou encore de transcriptions de conference calls. Pouvoir extraire automatiquement le sentiment véhiculé par ces textes représente un enjeu concret pour les professionnels de la finance, que ce soit pour alimenter des stratégies de trading algorithmique ou simplement pour synthétiser l’actualité d’une entreprise.

Notre projet s’inscrit dans ce cadre. L’idée de départ était assez simple : prendre un jeu de données de référence dans le domaine, y entraîner plusieurs modèles allant du plus basique au plus sophistiqué, et observer concrètement comment les performances évoluent d’une génération d’approches NLP à la suivante. Nous voulions aussi aller au-delà du simple benchmarking en construisant un outil de démonstration complet, avec un dashboard web permettant de tester les modèles en conditions réelles — y compris via saisie vocale et analyse en temps réel de l’actualité boursière.

Le choix des trois familles de modèles n’est pas arbitraire. Les approches classiques à base de TF-IDF représentent l’état de l’art d’il y a une dizaine d’années. Les réseaux récurrents comme le BiLSTM ont marqué la transition vers le deep learning en NLP. Et les transformers, depuis BERT en 2018, ont fondamentalement changé la donne en introduisant le pré-entraînement massif suivi du fine-tuning sur la tâche cible.

2. Dataset

2.1 FinancialPhraseBank

Le jeu de données que nous avons utilisé est le FinancialPhraseBank, publié par Malo et al. en 2014 dans le Journal of the Association for Information Science and Technology. Il contient 4846 phrases en anglais extraites de communiqués financiers d’entreprises européennes, principalement finlandaises.

Chaque phrase a été annotée par un panel de 16 experts en finance. La consigne d’annotation était la suivante : “Cette phrase, du point de vue d’un investisseur, donne-t-elle une impression positive, négative ou neutre de l’entreprise mentionnée ?” Seules les phrases pour lesquelles une majorité d’annotateurs (plus de 50%) s’est accordée sur une classe sont incluses dans le dataset final.

2.2 Distribution des classes

La distribution des classes présente un déséquilibre notable. La classe neutre domine avec 2 879 phrases (59,4%), suivie de la classe positive avec 1 363 phrases (28,1%). La classe négative ne représente que 604 phrases, soit 12,5% du total. Ce déséquilibre a des conséquences directes sur l'évaluation des modèles : un classificateur naïf qui prédit systématiquement "neutre" obtiendrait déjà 59% d'accuracy, mais un macro-F1 proche de zéro.

Classe	Nombre	Part
Neutral	2 879	59,4%
Positive	1 363	28,1%
Negative	604	12,5%
Total	4 846	100%

TABLE 1 – Distribution des classes dans le FinancialPhraseBank.

C'est précisément pour cette raison que nous avons retenu le macro-F1 comme métrique principale plutôt que l'accuracy.

L'accuracy se définit simplement comme le rapport entre le nombre de prédictions correctes et le nombre total d'exemples. Sur un dataset aussi déséquilibré que le nôtre, cette métrique est trompeuse : un modèle qui prédirait systématiquement "neutre" sans jamais détecter une phrase positive ou négative obtiendrait une accuracy de 59,4% — ce qui semble acceptable à première vue, mais correspond en réalité à un modèle parfaitement inutile.

Le macro-F1 corrige ce biais en calculant, pour chaque classe c , son F1-score individuel (moyenne harmonique de la précision et du rappel sur cette classe), puis en faisant la moyenne non pondérée de ces scores :

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c \quad \text{avec} \quad F1_c = \frac{2 \cdot \text{Précision}_c \cdot \text{Rappel}_c}{\text{Précision}_c + \text{Rappel}_c} \quad (1)$$

Le mot clé ici est *non pondérée* : chacune des trois classes compte pour un tiers dans le score final, indépendamment de sa fréquence dans le dataset. La classe négative (12,5% des données) pèse donc autant que la classe neutre (59,4%). Un modèle qui obtient un recall de 0% sur la classe négative — comme c'est le cas pour notre BiLSTM — sera sévèrement pénalisé, même s'il classe correctement la quasi-totalité des exemples neutres.

Pour illustrer concrètement : le BiLSTM obtient 63,7% d'accuracy mais seulement 37,3% de macro-F1. Cet écart de plus de 26 points révèle qu'il ne détecte pratiquement jamais les phrases négatives. L'accuracy masquait ce problème ; le macro-F1 le met en évidence.

C'est pourquoi nous l'avons retenu comme critère de comparaison principal entre tous nos modèles.

2.3 Split des données

Nous avons divisé les données en trois ensembles avec un split stratifié (seed=42) pour garantir la reproductibilité et une représentation proportionnelle de chaque classe dans chaque sous-ensemble.

Split	Samples	Part
Train	3 391	70%
Validation	485	10%
Test	970	20%

TABLE 2 – Répartition du split train/val/test.

Ce même split est utilisé pour tous les modèles sans exception, ce qui garantit que les comparaisons sont équitables.

3. Modèles développés

3.1 Approche classique : TF-IDF + classifieurs linéaires

La première famille de modèles que nous avons implémentée représente l'approche la plus traditionnelle en NLP. Le principe est de transformer chaque texte en un vecteur creux via TF-IDF (Term Frequency-Inverse Document Frequency), puis d'entraîner un classifieur linéaire sur ces représentations.

Nous avons utilisé des unigrammes et des bigrammes, avec un vocabulaire plafonné à 20 000 features et une pondération TF sublinéaire. Deux classifieurs ont été testés : la régression logistique (avec régularisation L2 et solver lbfgs) et le Naive Bayes multinomial (alpha=0,1).

L'avantage principal de ces modèles est leur rapidité : l'entraînement prend moins d'une seconde sur CPU. Ils sont aussi relativement interprétables. Leur limite fondamentale est qu'ils traitent le texte comme un sac de mots, ignorant complètement l'ordre des termes et le contexte dans lequel ils apparaissent.

3.2 Deep Learning : BiLSTM

Pour la deuxième approche, nous avons implémenté un réseau de neurones récurrent bidirectionnel. L'architecture se compose d'une couche d'embedding (dimension 64, vo-

cabulaire de 15 000 tokens), suivie de deux couches BiLSTM (128 unités chacune), d'une couche dense, et enfin d'un softmax pour la classification.

L'entraînement a été effectué sur 20 epochs avec un batch size de 32, en utilisant l'optimiseur Adam. Nous avons mis en place un early stopping sur la loss de validation pour éviter le sur-apprentissage. Le temps d'entraînement est d'environ 40 secondes, que ce soit sur CPU ou GPU.

Sur le papier, le BiLSTM devrait capturer l'ordre des mots et les dépendances séquentielles mieux que l'approche TF-IDF. En pratique, les résultats ont été décevants, comme nous le verrons dans la section résultats. Le modèle n'a pas réussi à généraliser correctement, en particulier sur la classe négative.

3.3 Transformers : BERT, DistilBERT, FinBERT

La troisième et dernière famille de modèles repose sur l'architecture Transformer, introduite par Vaswani et al. en 2017 et popularisée par BERT (Devlin et al., 2019). Le principe est de partir d'un modèle pré-entraîné sur un très large corpus de textes génériques, puis de le fine-tuner sur notre tâche spécifique.

Nous avons fine-tuné trois variantes. BERT (**base-uncased**) est le modèle original avec 110 millions de paramètres, pré-entraîné sur BookCorpus et Wikipedia. DistilBERT est une version distillée de BERT, 40% plus légère (66 millions de paramètres) tout en conservant environ 97% de ses performances sur les benchmarks génériques. FinBERT est une version de BERT qui a subi un pré-entraînement supplémentaire sur un corpus de textes financiers (rapports d'analystes, earnings calls, communiqués de presse). Ce pré-entraînement additionnel lui permet de comprendre les nuances propres au vocabulaire financier, comme les termes *restructuring*, *write-down* ou *impairment charge* qui ont une connotation négative spécifique au domaine.

Pour les trois modèles, les hyperparamètres de fine-tuning sont identiques : 5 epochs, learning rate de $2e-5$, avec early stopping sur la performance de validation. Nous avons dû mettre en place plusieurs optimisations GPU pour faire tourner l'entraînement sur notre carte graphique (RTX 5070 Laptop, 8 Go de VRAM) : précision mixte fp16, gradient checkpointing pour BERT et FinBERT (qui sont plus gourmands en mémoire), gradient accumulation, et un plafonnement de l'utilisation VRAM à 85% pour éviter les erreurs de mémoire.

4. Résultats et benchmark

4.1 Tableau comparatif

Rang	Modèle	Accuracy	Macro-F1	Type	Temps
1	FinBERT	88,1%	87,7%	Transformer	82s
2	DistilBERT	86,8%	86,0%	Transformer	51s
3	BERT	85,5%	83,8%	Transformer	111s
4	TF-IDF + NaiveBayes	71,3%	62,7%	Classical	<1s
5	TF-IDF + LogReg	72,4%	60,1%	Classical	<1s
6	BiLSTM	63,7%	37,3%	Deep Learning	41s

TABLE 3 – Benchmark des 6 modèles sur le split de test (970 exemples).

4.2 Leaderboard interactif

En complément du tableau statique ci-dessus, le dashboard intègre un leaderboard interactif mis à jour automatiquement à chaque exécution du script `build_leaderboard.py`. Ce script agrège les résultats de tous les modèles enregistrés dans `results/leaderboard.json` et génère simultanément un fichier CSV, un fichier JSON et un tableau Markdown. Le leaderboard du dashboard affiche, pour chaque modèle, l’accuracy, le macro-F1, le weighted-F1, le type d’architecture et le temps d’entraînement, triés par macro-F1 décroissant.

Cette conception facilite la comparaison visuelle entre les trois familles de modèles et rend immédiatement lisible la hiérarchie des performances : les transformers dominent nettement, FinBERT en tête, tandis que le BiLSTM se retrouve en bas du classement malgré une architecture pourtant plus sophistiquée que les baselines classiques.

4.3 Analyse des résultats

Les résultats montrent une hiérarchie assez nette. FinBERT arrive en tête avec un macro-F1 de 87,7%, ce qui confirme l’apport du pré-entraînement spécifique au domaine financier. L’écart avec BERT générique (+3,9 points de macro-F1) est significatif et illustre bien l’intérêt d’adapter le modèle au vocabulaire cible.

Un résultat surprenant est la performance de DistilBERT (86,0%), qui dépasse BERT complet (83,8%) malgré un nombre de paramètres inférieur de 40%. Notre hypothèse est que sur un petit dataset comme le FinancialPhraseBank, un modèle plus compact est moins susceptible d’overfitter. DistilBERT est aussi nettement plus rapide à fine-tuner (51s contre 111s pour BERT).

Les baselines classiques montrent un phénomène intéressant : leur accuracy est correcte (71–72%) mais leur macro-F1 est bien plus faible (60–63%). Cet écart s’explique par le fait que ces modèles prédisent majoritairement “neutre”, ce qui leur donne une bonne accuracy grâce à la prévalence de cette classe, mais les fait échouer sur les classes minoritaires.

Le cas le plus parlant est celui du BiLSTM, qui obtient le pire macro-F1 du benchmark

(37,3%). En analysant les prédictions, nous avons constaté qu'il obtenait un recall de 0% sur la classe négative. Sans pré-entraînement et avec seulement 5 000 exemples d'entraînement, le réseau récurrent n'a tout simplement pas assez de données pour apprendre les patterns de chaque classe, surtout la classe minoritaire.

4.4 Enseignements

Le principal enseignement de ce benchmark est le rôle crucial du pré-entraînement dans un contexte de faible volume de données. Les transformers pré-entraînés surpassent largement les approches non pré-entraînées, et le pré-entraînement spécialisé (FinBERT) apporte un gain supplémentaire. Ce constat est cohérent avec la littérature récente en NLP : le paradigme “pre-train then fine-tune” est particulièrement efficace quand le dataset cible est petit.

5. Dashboard et fonctionnalités

5.1 Architecture du dashboard

Pour rendre nos modèles accessibles et démontrables, nous avons développé un dashboard web en utilisant Streamlit. L'interface se compose de huit pages accessibles via un menu de navigation latéral : Analyze, Batch Analysis, Live News, Leaderboard, Dataset, Methodology, Rendu et About. Le design a été entièrement personnalisé avec un système de CSS injecté, comprenant une sidebar sombre, des cartes avec bordures et ombres, et une typographie Inter pour un rendu professionnel.

Le chargement des modèles est mis en cache grâce à `st.cache_resource`, ce qui évite de recharger les poids à chaque interaction et garantit une expérience fluide même lors de l'analyse de nombreuses phrases successives.

5.2 Analyse individuelle : saisie textuelle et saisie vocale

La page principale d'analyse permet à l'utilisateur de soumettre une phrase financière de deux façons équivalentes : par saisie textuelle classique ou par dictée vocale. Dans les deux cas, le flux de traitement est identique — la phrase est transmise au modèle sélectionné et la prédiction est affichée avec les scores de confiance pour les trois classes (positive, neutre, négative).

5.2.1 Saisie textuelle

L'utilisateur tape directement une headline financière dans le champ de saisie. Après validation, le modèle actif analyse la phrase et retourne la classe prédite accompagnée

d'un score de confiance pour chaque label. La sélection du modèle se fait via un sélecteur situé directement sur la page, à côté du bouton d'analyse, ce qui permet de changer de modèle sans naviguer ailleurs dans l'interface.

5.2.2 Saisie vocale

Pour aller au-delà de la saisie clavier, nous avons intégré une entrée vocale permettant à l'utilisateur de dicter une headline à la manière des interfaces conversationnelles modernes. L'implémentation repose sur la Web Speech API du navigateur (compatible Chrome et Edge). Une fois le bouton microphone activé, la transcription s'effectue en temps réel et le texte reconnu est injecté automatiquement dans le champ de saisie. L'analyse est ensuite déclenchée de façon identique à la saisie textuelle : le même pipeline de prédiction est appelé, avec le même affichage des résultats. L'utilisateur n'a donc pas à distinguer les deux modes — le résultat est toujours une prédiction de sentiment sur la phrase transmise, quelle que soit la modalité d'entrée.

Le principal défi technique résidait dans l'architecture de Streamlit : tous les composants HTML personnalisés sont rendus dans des iframes sandboxées qui n'ont pas accès au microphone du navigateur. Notre solution finale utilise `components.html()` comme bootstrap pour injecter un élément `<script>` dans le document parent de Streamlit, qui s'exécute dans le bon contexte JavaScript et peut ainsi accéder au microphone. La transcription est ensuite injectée dans le textarea via le setter natif du prototype `HTMLTextAreaElement`, suivi d'événements synthétiques pour que Streamlit détecte la nouvelle valeur.

5.3 Comparaison multi-modèles et rôle de FinBERT

L'une des fonctionnalités les plus pédagogiques du dashboard est le mode de comparaison multi-modèles. Une fois une phrase saisie — que ce soit par dictée ou par saisie manuelle — l'utilisateur peut choisir de faire analyser cette même phrase par les trois transformeurs simultanément (BERT, DistilBERT et FinBERT), sans avoir à resaisir le texte ni à naviguer entre plusieurs pages.

Les trois prédictions sont affichées côte à côte, ce qui permet de visualiser directement les divergences entre les modèles. Dans la majorité des cas, les trois transformeurs s'accordent, ce qui renforce la confiance dans la prédiction. Mais sur des phrases ambiguës ou formulées avec un vocabulaire financier technique, des écarts apparaissent : FinBERT donnera fréquemment la réponse la plus précise, là où BERT et DistilBERT peuvent hésiter ou se tromper.

Ce phénomène s'explique directement par le pré-entraînement de FinBERT. Un modèle pré-entraîné sur des textes génériques comme Wikipedia ne sait pas, a priori, que des termes comme *impairment charge*, *write-off* ou *profit warning* sont systématiquement as-

sociés à des nouvelles négatives. FinBERT, ayant ingéré des millions de pages de rapports financiers et d’earnings calls lors de son pré-entraînement additionnel, a internalisé ces associations sémantiques spécifiques au domaine. Il produit donc des représentations vectorielles mieux adaptées au vocabulaire de la finance, ce qui se traduit par un macro-F1 supérieur de près de 4 points à celui de BERT générique sur notre dataset.

En pratique, le mode comparaison multi-modèles est particulièrement instructif pour illustrer ce point : sur une phrase comme *“The company recorded a significant write-down on its assets”*, FinBERT attribuera sans hésitation une forte probabilité négative, tandis que BERT ou DistilBERT pourraient être moins affirmatifs. Ce différentiel rend FinBERT à la fois le modèle le plus performant et le plus rationnel dans le contexte de ce projet — celui dont les prédictions sont les mieux ancrées dans la sémantique financière réelle.

5.4 Explicabilité avec LIME

Après chaque prédiction — en mode analyse individuelle comme en mode comparaison multi-modèles — le dashboard affiche automatiquement une analyse d’explicabilité basée sur LIME (*Local Interpretable Model-Agnostic Explanations*, Ribeiro et al., 2016).

LIME fonctionne en générant des perturbations locales de la phrase d’entrée (en masquant ou remplaçant des mots) et en observant comment la prédiction du modèle évolue. Les mots dont la suppression fait le plus varier le score de confiance sont ceux identifiés comme les plus influents. Le résultat est une attribution de poids à chaque mot de la phrase, permettant de répondre à la question : *quels mots ont conduit le modèle à prédire cette classe, et dans quelle mesure ?*

Dans le dashboard, cette information est visualisée de deux manières complémentaires. D’une part, un surlignage coloré de la phrase originale : les mots qui ont poussé vers la classe prédite apparaissent en vert, ceux qui ont joué contre elle en rouge, avec une intensité proportionnelle à leur poids LIME. D’autre part, un diagramme à barres horizontales affiche explicitement le score d’importance de chaque mot, ce qui permet de comparer quantitativement leur influence respective.

Ce mécanisme d’explicabilité est particulièrement révélateur lorsqu’il est appliqué à FinBERT. On observe que ce modèle identifie correctement les marqueurs sémantiques du domaine financier — des mots comme *restructuring*, *losses* ou *exceeded expectations* reçoivent des poids élevés et cohérents avec leur connotation réelle. Cela confirme, au niveau des mots individuels, que FinBERT ne se contente pas d’un pattern matching superficiel mais qu’il a bien intégré la sémantique financière lors de son pré-entraînement. En comparant les attributions LIME de FinBERT avec celles de BERT ou DistilBERT sur la même phrase, l’utilisateur peut constater directement pourquoi FinBERT est le modèle le plus précis : il détecte les bons signaux là où les modèles génériques peuvent se laisser

influencer par des mots moins pertinents.

5.5 Batch Analysis

La page Batch Analysis permet de traiter un fichier CSV complet. L'utilisateur uploadé un fichier contenant une colonne de headlines, et le système classe chaque ligne avec le modèle actif. Les résultats sont présentés sous forme de métriques synthétiques (nombre par classe), de graphiques (camembert de distribution, histogramme de confiance), d'un tableau détaillé, et d'un bouton de téléchargement CSV.

5.6 Live News via Yahoo Finance

Pour aller au-delà des données statiques, nous avons intégré une fonctionnalité d'analyse en temps réel. L'utilisateur entre un ticker boursier (par exemple AAPL pour Apple) et le système utilise la librairie `yfinance` pour récupérer les dernières headlines d'actualité associées à ce titre. Chaque headline est ensuite classifiée par le modèle actif, et un score de sentiment global est calculé en faisant la différence entre la proportion de headlines positives et négatives.

Le principal défi technique a été l'extraction des titres depuis la structure de données renvoyée par l'API `yfinance`, qui varie selon les versions et les tickers. Nous avons implémenté un parsing robuste qui gère les différents formats possibles.

6. Difficultés rencontrées

Tout au long du projet, nous avons fait face à un certain nombre de difficultés techniques que nous souhaitons documenter ici.

La première difficulté concernait la gestion de la mémoire GPU. Le fine-tuning de BERT et FinBERT (110 millions de paramètres chacun) sur une carte avec 8 Go de VRAM nécessitait des optimisations spécifiques. Nous avons dû activer la précision mixte `fp16`, le gradient checkpointing (qui échange du temps de calcul contre de la mémoire), la gradient accumulation pour simuler des batch sizes plus grands, et un plafonnement de l'allocation mémoire à 85% de la VRAM disponible. Sans ces mesures, l'entraînement échouait systématiquement avec des erreurs CUDA out-of-memory.

La deuxième difficulté était liée à l'installation de PyTorch avec support CUDA sur Windows. La commande standard `pip install torch` installe une version CPU-only. Il a fallu configurer un index de paquets spécifique (`pytorch-cu128`) dans le fichier `pyproject.toml` via la directive `[tool.uv.sources]` pour obtenir une version compatible avec notre driver CUDA.

La troisième difficulté, et probablement la plus chronophage, a été l'implémentation de l'entrée vocale dans Streamlit. Nous avons exploré et abandonné plusieurs pistes avant de trouver la solution d'injection dans le contexte parent. Le problème est que les iframes Streamlit ont un attribut `sandbox` qui bloque l'accès au microphone, et que les permissions du navigateur exigent que la création du `SpeechRecognition` se fasse dans le bon contexte JavaScript avec un geste utilisateur explicite.

Enfin, la synchronisation entre les modifications JavaScript du DOM et l'état interne de Streamlit a posé des problèmes subtils. Quand on modifie la valeur d'un textarea via JavaScript, Streamlit ne détecte pas automatiquement le changement. Nous avons dû utiliser le setter natif du prototype `HTMLTextAreaElement`, dispatcher des événements synthétiques `input` et `change`, et utiliser des callbacks `on_click` plutôt que des affectations directes au `session_state` pour contourner les restrictions de Streamlit sur la modification de l'état après l'instanciation d'un widget.

7. Conclusion

Ce projet nous a permis de concrétiser et comparer trois générations d'approches NLP sur une tâche réelle de classification de sentiment financier. Le résultat principal est sans ambiguïté : FinBERT domine le benchmark avec un macro-F1 de 87,7%, confirmant que le pré-entraînement spécifique au domaine est le facteur le plus déterminant pour la performance sur un petit dataset spécialisé. L'analyse d'explicabilité LIME vient renforcer ce constat au niveau des mots : FinBERT identifie les bons signaux sémantiques là où les modèles génériques peuvent se laisser égarer.

Au-delà du benchmarking, le développement du dashboard Streamlit a été l'occasion de confronter nos modèles à des cas d'usage concrets (analyse en temps réel, traitement par lot, saisie vocale, explicabilité) et de résoudre des problèmes d'ingénierie logicielle non triviaux (gestion GPU, intégration Web Speech API, synchronisation JavaScript-Python).

Parmi les pistes d'amélioration, on peut envisager l'élargissement du dataset d'entraînement (SEC filings, earnings call transcripts), l'expérimentation avec des modèles plus récents comme DeBERTa ou des approches few-shot à base de LLMs, et le déploiement du dashboard sur une infrastructure cloud avec une API REST pour un accès plus large.

Références

1. Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). Good debt or bad debt : Detecting semantic orientations in economic texts. *Journal of the Association for Information Science and Technology*, 65(4), 782–796.
2. Araci, D. (2019). FinBERT : Financial Sentiment Analysis with Pre-trained Language Models. *arXiv preprint arXiv :1908.10063*.
3. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT*.
4. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*.
5. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT : smaller, faster, cheaper and lighter. *arXiv preprint arXiv :1910.01108*.
6. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why Should I Trust You?” : Explaining the Predictions of Any Classifier. *KDD*.